🌿 PlantVision AI

> Cross-platform plant identification app powered by AI — identify any plant instantly from your camera or gallery, with detailed information in 10+ Indian languages.

📷 Screenshots

<!-- Add your screenshots here after uploading them to the repo --> <!-- Example: <p align="center">   </p> -->

> *Add screenshots to a `/screenshots` folder in the repo and uncomment the block above.*

🚀 Features

- **AI-powered plant identification** using PlantNet API + custom TensorFlow/Keras MobileNetV2 model
- **Real-time camera capture** and **gallery upload** support
- **Cross-platform** — runs on Android, iOS, Web, and Desktop from a single codebase
- **Detailed plant information**: scientific names, ecological data, habitat, and disease detection
- **Authentic local names in 10+ Indian languages** via Wikidata & Wikipedia APIs (not translation — real vernacular names)
- **Offline-ready architecture** with SQLite local caching

🛠️ Tech Stack

Layer	Technology
Frontend	Flutter (Dart)
Backend	Python FastAPI
AI / ML	TensorFlow, Keras, MobileNetV2
Plant ID	PlantNet API

Layer	Technology
Biodiversity Data	GBIF API
Local Names	Wikidata SPARQL, Wikipedia API
Database	SQLite (local), Firebase (cloud sync)
Platforms	Android, iOS, Web, Desktop

📁 Project Structure

```
PlantVision-AI/
├── backend/                 # FastAPI backend
│   ├── main.py              # App entry point
│   ├── routes/              # API route handlers
│   │   ├── identify.py      # Plant identification endpoint
│   │   ├── details.py       # Plant details & local names
│   │   └── disease.py       # Disease detection endpoint
│   ├── services/            # Business logic
│   │   ├── plantnet.py      # PlantNet API integration
│   │   ├── gbif.py          # GBIF biodiversity data
│   │   ├── wikidata.py      # Wikidata SPARQL queries
│   │   └── ml_model.py      # TensorFlow/Keras model inference
│   ├── models/              # TF/Keras model files
│   ├── database/            # SQLite schema & helpers
│   └── .env.example         # Required environment variables
│
├── frontend/                # Flutter app
│   ├── lib/
│   │   ├── main.dart
│   │   ├── screens/         # UI screens
│   │   ├── widgets/         # Reusable components
│   │   └── services/        # API service layer
│   └── pubspec.yaml
│
└── README.md
```

⚙️ Setup & Installation

Prerequisites

- Python 3.9+
- Flutter SDK 3.x
- Git

1. Clone the repository

```bash
git clone https://github.com/sushg1421/PlantVision-AI.git
cd PlantVision-AI
```

2. Backend setup

```bash
cd backend

# Create virtual environment
python -m venv venv
source venv/bin/activate        # On Windows: venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Set up environment variables
cp .env.example .env
# Edit .env and add your API keys (see below)

# Run the server
uvicorn main:app --reload
```

3. Environment variables (`.env`)

```env
PLANTNET_API_KEY=your_plantnet_api_key
GEMINI_API_KEY=your_gemini_api_key
GBIF_API_KEY=your_gbif_api_key
```

4. Flutter frontend setup

```bash
cd frontend
flutter pub get
flutter run                    # Runs on connected device/emulator
```

For specific platforms:

```bash
flutter run -d chrome          # Web
flutter run -d windows         # Desktop (Windows)
flutter build apk              # Android APK
```

🔌 API Endpoints

Method	Endpoint	Description
`POST`	`/identify`	Identify plant from image
`GET`	`/plant/{id}`	Get detailed plant info
`GET`	`/plant/{id}/local-names`	Get names in Indian languages
`POST`	`/disease`	Detect plant disease from image

🌐 How It Works

```
User uploads image
       ↓
PlantNet API → returns top species matches
       ↓
TensorFlow/Keras MobileNetV2 → confirms identification
       ↓
GBIF API → ecological & biodiversity data
       ↓
```

```
Wikidata SPARQL + Wikipedia API → authentic local names in 10+ languages
       ↓
Result displayed in Flutter UI
```

🌍 Supported Languages for Local Names

Hindi · Kannada · Tamil · Telugu · Malayalam · Marathi · Bengali · Gujarati · Punjabi · Odia · Urdu

> Local names are sourced directly from Wikidata and Wikipedia — not translated — ensuring authentic vernacular names rather than transliterations.

📊 Model Details

- **Architecture:** MobileNetV2 (transfer learning)
- **Framework:** TensorFlow / Keras
- **Task:** Multi-class plant species classification
- **Input:** 224×224 RGB images
- **Training data:** PlantVillage dataset + custom collected samples

---

🧑‍💻 Author

Sushanth H N

- GitHub: @sushg1421
- Email: sushanthg861@gmail.com
- LinkedIn: linkedin.com/in/sushanth-h-n

📄 License

<p align="center">Made with 🌿 by Sushanth H N</p>